

Recursion Lab

CSE 114 INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING



Recursion Exercise 1 of 3

Pretend that Java has + and – but does not have * or /.

We want to write a recursive function `mult(a, b)` :

```
/*  
    a and b are non-negative integers.  
    Return: the product of a and b  
*/  
public static int mult(int a, int b){
```

We want to use a recursive algorithm:

If b is 0, $a*b$ is 0.

If $b > 0$, $a*b$ is $a*(b-1)+a$

Once this is working try to get it to work with possible negative arguments as well.

Exercise 2 of 3

Write a recursive version of the `stringTimes` function from the midterm:

```
/*
    stringTimes
    n should be greater than or equal to 0.
    Returns a string which consists of n copies
    of s concatenated together.

    Example: stringTimes("age", 4) is "ageageageage".
*/

public static String stringTimes(String s, int n);
```

Exercise 3 of 3

Write a recursive function that returns the concatenation of all the strings in an array:

```
/*  
    a is an array of strings.  
    Returns the concatenation of all the strings  
    between a[low] and a[high]  
    Example: if a = {"hello ", "out ", "there"}, then  
    concat(a, 0, 2) should return "hello out there".  
*/  
public static String concat(String [] a, int low, int high);
```

Recursive array functions usually work by separating out the first element of the array from the rest of the array. The rest of the array can be concatenated recursively, then the first element added on.